

Class constraints aula....

Qual será o tipo das funções `elem` e `max` ?

```
> elem 3 [1,2,3,4,5]
True
> elem 7 [1,2,3,4,5]
False
> max 35 28
35
> max 5.6 10.7
10.7
```

```
elem :: Eq a => a -> [a] -> Bool
```

Porque se usa a função `(==)` na sua implementação.

```
max :: Ord a => a -> a -> a
```

Porque se usa a função `(<)` na sua implementação.

Mais alguns operadores do Prelude

Lógicos: `&&` (conjunção), `||` (disjunção), `not` (negação)

Numéricos: `+`, `-`, `*`, `/` (divisão de reais), `^` (exponenciação com inteiros),
`div` (divisão inteira), `mod` (resto da divisão inteira), `abs` (módulo)
`**` (exponenciações com reais), `log`, `sin`, `cos`, `tan`, ...

Relacionais: `==` (igualdade), `/=` (desigualdade), `<`, `<=`, `>`, `>=`

Condicional: `if ... then ... else ...`

```
graph TD; B[Bool] --> P1[...]; a[a] --> P2[then ...]; a --> P3[else ...];
```

Definição de funções

- A definição de funções faz-se através de uma sequência de equações da forma:

nome $arg_1 arg_2 \dots arg_n = expressão$

- O nome das funções começa sempre por letra minúscula (ou *underscore*).
- Quando se define uma função podemos indicar o seu tipo. No entanto, isso não é obrigatório.
- O tipo de cada função é inferido automaticamente pelo compilador.
- O compilador infere o tipo mais geral que se pode associar à função. No entanto, é possível atribuir à função um tipo mais específico.

```
swap :: (a,b) -> (b,a)  
swap (x,y) = (y,x)
```

```
troca :: (Int,Char) -> (Char,Int)  
troca (x,y) = (y,x)
```

- É boa prática de programação indicar o tipo das funções definidas no seus programas Haskell.

Exemplos

- Uma função de nome `fun` que recebe dois inteiros e, caso sejam ambos positivos, soma-os; caso contrário multiplica-os.

```
fun :: Int -> Int -> Int
fun x y = if x>0 && y>0 then x+y else x*y
```

- Uma função de nome `media` que recebe uma lista de inteiros e calcula a sua média aritmética.

```
media :: [Int] -> Int
media l = div (sum l) (length l)
```

Alternativamente poderíamos escrever:

```
media l = (sum l) `div` (length l)
```

Quando uma função é binária, podemos escrevê-la de forma **infixa** (entre os seus argumentos) colocando o seu nome entre ```

Exercícios

```
['a','b','c']  
( 'a','b','c' )  
[(False,'0'),(True,'1')]  
([False,True],['0','1'])  
[tail, reverse, take 2]
```

Qual será o tipo destas expressões ?

```
second xs = head (tail xs)  
pair x y = (x,y)  
double x = x*2  
palindrome xs = reverse xs == xs  
twice f x = f (f x)
```

Qual será o tipo destas funções ?

Teste as suas respostas no GHCi.

Exercícios

```
['a','b','c']
```

:: [Char]

```
('a','b','c')
```

:: (Char,Char,Char)

```
[(False,'0'),(True,'1')]
```

:: [(Bool,Char)]

```
([False,True],['0','1'])
```

:: ([Bool],[Char])

```
[tail, reverse, take 2]
```

:: [[a] -> [a]]

Exercícios

```
second xs = head (tail xs)
pair x y = (x,y)
double x = x*2
palindrome xs = reverse xs == xs
twice f x = f (f x)
```

`:: [a] -> a`

`:: a -> b -> (a,b)`

`:: Num a => a -> a`

`:: Eq a => [a] -> Bool`

`:: (a->a) -> a -> a`

Módulos

- Um programa Haskell está organizado em **módulos**.
- Cada módulo é uma coleção de definições num **ambiente fechado**.
- Um módulo pode **exportar** todas ou só algumas das suas definições. (...)

```
module Nome (...) where  
  
...definições...
```

- Um módulo constitui um **componente de software** e dá a possibilidade de gerar bibliotecas de funções que podem ser **reutilizadas** em diversos programas.
- Para se utilizarem declarações feitas noutros módulos, que não o `Prelude`, é necessário primeiro fazer a sua importação através da instrução:

```
import Nome
```


Módulos

Exemplo: Muitas funções sobre caracteres estão definidas no módulo `Data.Char`.

```
module Exemplo where
import Data.Char
letra :: Int -> Char
letra n = if (n>=65 && n<=90) || (n>=97 && n<=122)
          then chr n
          else '-'
numero :: Int -> Char
numero n = if (n>=48 && n<=57)
            then chr n
            else '-'
```

`chr` é uma função do módulo `Data.Char` que dado o código ASCII de um carácter devolve o respectivo carácter.

```
> letra 100
'd'
> letra 5
'-'
> numero 3
'-'
> numero 55
'7'
```

ASCII (American Standard Code for Information Interchange)

Codificação standard dos caracteres de 8 bits baseada no alfabeto inglês.

A tabela vai de 0 a 127.

...	...
9 - '\t'	65 - 'A'
10 - '\n'	66 - 'B'
...	...
32 - ' '	90 - 'Z'
...	...
48 - '0'	97 - 'a'
49 - '1'	98 - 'b'
...	...
57 - '9'	122 - 'z'
	...

Comentários

O código Haskell pode ser comentado de duas formas:

- `--` O texto que aparecer a seguir a `--` até ao final da linha é ignorado.
- `{- ... -}` O texto que estiver entre `{-` e `-}` não é avaliado pelo interpretador. Podem ser várias linhas.

```
module Exemplo where
import Data.Char

-- letra recebe um ASCII e devolve o caracter que lhe corresponde
-- se for uma letra; caso contrário dá '-'
letra :: Int -> Char
letra n = if (n>=65 && n<=90) || (n>=97 && n<=122)
           then chr n
           else '-'          -- porque não é uma letra
{- numero recebe um ASCII e devolve o caracter que lhe corresponde
se for um algarismo; caso contrário dá '-' -}
numero :: Int -> Char
numero n = if (n>=48 && n<=57)
            then chr n
            else '-'
```

Tipos sinónimos

Em Haskell é possível renomear tipos através de declarações da forma

```
type Nome  $p_1 \dots p_n$  = tipo
```

Exemplos:

```
type Coordenada = (Float,Float)
distancia :: Coordenada -> Coordenada -> Float
distancia (x1,y1) (x2,y2) = sqrt ((x2-x1)^2+(y2-y1)^2)
```

```
type Triplo a = (a,a,a)
multri :: Triplo Int -> Int
multri (x,y,z) = x*y*z
```

Strings

O tipo `String` é um tipo sinónimo já definido no `Prelude`.

```
type String = [Char]
```

Os valores do tipo `String` podem ser escritos como [sequências de caracteres entre aspas](#).

```
> ['o','l','a']  
"ola"  
> length "ola"  
3  
> reverse "ola"  
"alo"
```

```
type Numero = Integer  
type Nome = String  
type Curso = String  
type Aluno = (Numero, Nome, Curso)  
type Turma = [Aluno]
```

Declarações locais

- Todas as declarações que vimos até aqui são globais. Ou seja, são visíveis no módulo onde estão.
- Muitas vezes é útil reduzir o âmbito de uma declaração, para tornar o código mais legível.
- O Haskell permite fazer

```
fun x = let v = x*x + x^10  
      in x + v + 4*v
```

- [declarações locais a uma expressão](#), utilizando a construção **let ... in ...**
- [declarações locais a uma equação](#), utilizando a construção **where ...**

```
exemplo x y = (a,b)  
      where a = x+y  
           b = sum [1..a]
```

Declarações locais

```
dis :: Coordenada -> Coordenada -> Float
dis (x1,y1) (x2,y2) = let a = (x2-x1)^2
                        b = (y2-y1)^2
                        in sqrt (a+b)
```

```
dist :: Coordenada -> Coordenada -> Float
dist (x1,y1) (x2,y2) = sqrt (a+b)
  where a = (x2-x1)^2
        b = (y2-y1)^2
```

```
> dis (2,4.3) (-1,7.5)
4.386342
> dist (2,4.3) (-1,7.5)
4.386342
> a
error: Variable not in scope: a
```

Também é possível definir localmente funções com argumentos .

Layout

O Haskell não necessita de marcas para delimitar as diversas declarações que constituem um programa. A **indentação do texto** (isto é, o espaço entre a margem e o início do texto) tem um significado preciso:

- Se uma linha começa mais à frente do que começou a linha anterior, então ela deve ser considerada como a continuação da linha anterior.
- Se uma linha começa na mesma coluna que a anterior, então elas são consideradas definições independentes.
- Se uma linha começa mais atrás do que a anterior, então essa linha não pertence à mesma lista de definições.

As declarações das funções `dis` e `dist` começam na mesma coluna.

```
dis :: Coordenada -> Coordenada -> Float
dis (x1,y1) (x2,y2) = let a = (x2-x1)^2
                        b = (y2-y1)^2
                        in sqrt (a+b)
```

As declarações de `a` e `b` dentro do `let-in` começam na mesma coluna.

`where` começa numa coluna mais à frente porque é a continuação da declaração da equação.

```
dist :: Coordenada -> Coordenada -> Float
dist (x1,y1) (x2,y2) = sqrt (a+b)
    where a = (x2-x1)^2
           b = (y2-y1)^2
```

As declarações de `a` e `b` dentro do `where` começam na mesma coluna.

Tipos algébricos

Em Haskell podemos definir **novos tipos de dados** através de declarações da forma

```
data Nome  $p_1 \dots p_n =$  Construtor ... | ... | Construtor ...
```

- Os construtores são os **modos de construir valores do tipo** que está a ser declarado.
- O nome dos construtores começa sempre por **letra maiúscula**.

Temos exemplos destas definições na biblioteca Prelude:

```
data Bool = False | True
```

```
data Maybe a = Nothing | Just a
```

```
> :type True
True :: Bool
> :type Nothing
Nothing :: Maybe a
> :type Just "ola"
Just "ola" :: Maybe [Char]
> :type Just True
Just True :: Maybe Bool
> :type Just
Just :: a -> Maybe a
```

Tipos algébricos

Exemplo: podemos definir um novo tipo para representar cores

```
data Cor = Amarelo | Verde | Vermelho | Azul
    deriving (Show)
```

e definir uma função que testa se uma cor é fria

```
fria :: Cor -> Bool
fria Verde = True
fria Azul = True
fria x = False
```

```
> :type Verde
Verde :: Cor
> fria Verde
True
> fria Amarelo
False
```

Acrescentamos `deriving (Show)` para podermos visualizar os valores do novo tipo no interpretador.

O GHCi procura [de cima para baixo](#) a equação que pode usar para calcular o valor da expressão `(fria Amarelo)`. A primeira que encontra neste caso é a 3ª equação.

Se invertermos a ordem das equações, qual será a resposta do GHCi ao avaliar a expressão `(fria Verde)` ?

Tipos algébricos

Exemplo: podemos definir um novo tipo para representar pontos coloridos no plano cartesiano

```
data PontoC = PC Coordenada Cor
  deriving (Show)
```

```
> :type (PC (3.5,2.2) Azul)
(PC (3.5,2.2) Azul) :: PontoC
> :type (PC (-5,0.7) Verde)
(PC (-5,0.7) Verde) :: PontoC
> :type PC
PC :: Coordenada -> Cor -> PontoC
```

e definir uma função que calcula distância de um ponto colorido à origem do plano, assim

```
distOrigem :: PontoC -> Float
distOrigem (PC (x,y) c) = sqrt (x^2+y^2)
```

Ou então assim:

```
distOrigem :: PontoC -> Float
distOrigem (PC p c) = distancia p (0,0)
```